

Object Oriented Programming - Inheritance

In our previous sessions, we explore classes and objects. Now, we will delve into Inheritance, a fundamental pillar of OOP that allow use to create hierarchical relationships between classes, promoting code reuse and extansibility.

What is Inheritance?

Inheritance is a mechanism that allows a new class (the **child** or **derived** class) to inherit attributes and methods from an existing class (the **parent** or **base** class). Instead of building every class from scratch, you can create a foundation and build upon it.

- A **(child/derived) class** inherits, the attributes and behaviors of a **parent (base) class**.
- The programmer is able to use the features of the parent class without rewriting them.
- The child class can also:
 - **Add** its own attrubutes amd methods, or
 - **Override** methods from the parent class. The child class may optionally define its own attributes or overwrite the existing ones in the parent class.
- The `super()` function is used to access attribute and method from the parent class.

Benefits if Heritance

- **Promotes Code Reuse:** Avoids redundancy by sharing common lofic across related classes.
- **Model Real-World Relotionships:** Naturally represents "is-a" relationships (e.g., an Employee *is a* Person, a Dog *is an* Animal).
- **Simplifies Code Maintenance:** A change in the parent class sutomatically propagates to all child classes ensuring consistency.

Example 1: Basic Inheritance

```
#person class
class Person:
    '''Person class with name and age attributes'''
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f'{self.name}, {self.age} years old'
```

Here we define a **base class** Person with a constructor (`__init__`) and a `__str__` method.

```
#the following class inherits everything from Person
# including its constructor and __str__ method
```

Lecture 11

```
class Employee(Person):  
    pass
```

Employee doesn't define anything new - it simply inherits everything for Person

```
#test harness  
ilia = Employee('Ilia', 21 )      #uses its parent's constructor  
print(ilia)                       #uses its parent's __str__ method  
print(type(ilia))  
print(isinstance(ilia, Employee))  
print(issubclass(type(ilia), Person))
```

Even though we only created an Employee, all the features of Person came along "for free."

A More Complex Hierarchy

Let us model a more realistic hierarchy with Animal as our base class

The Base Class - Animal

```
class Animal:  
    '''Base class for all animals.'''  
  
    def __init__(self, name, species, age):  
        self.name = name  
        self.species = species  
        self.age = age  
        self.is_alive = True  
  
    def eat(self, food):  
        '''Animal eating behavior.'''  
        print(f'{self.name} is eating {food}')  
    def sleep(self):  
        '''Animal sleeping behavior.'''  
        print(f'{self.name} is sleeping')  
    def make_sound(self):  
        '''Generic animal sound - to be overridden.'''  
        print(f'{self.name} makes a sound')  
    def get_info(self):  
        '''Get basic animal information.'''  
        return {  
            'name': self.name,  
            'species': self.species,  
            'age': self.age,  
            'alive': self.is_alive  
        }
```

The Animal class defines **common attributes and behaviors** (eat, sleep, make sounds, etc.).

```
# test harness  
monster = Animal('Monster', 'Chupabra', 3)  
  
# Use inherited methods
```

Lecture 11

```
monster.eat('dog treats')
monster.sleep()
monster.make_sound()

print(f'Chupacabra info: {monster.get_info()}')
```

The child class - Dog (Inheritance and Overriding)

```
class Dog(Animal):
    '''Dog class inheriting from Animal.'''

    def __init__(self, name, breed, age, owner=None):
        # Call parent constructor
        super().__init__(name, 'Canis lupus', age)
        self.breed = breed
        self.owner = owner
        self.tricks = []

    def make_sound(self):
        '''Override parent method.'''
        print(f'{self.name} barks: Woof! Woof!')

    def fetch(self, item):
        '''Dog-specific behavior.'''
        print(f'{self.name} fetches the {item}')

    def learn_trick(self, trick):
        '''Teach the dog a new trick.'''
        self.tricks.append(trick)
        print(f'{self.name} learned to {trick}')

    def perform_trick(self, trick):
        '''Perform a learned trick.'''
        if trick in self.tricks:
            print(f'{self.name} performs {trick}')
        else:
            print(f'{self.name} doesn\'t know how to {trick}')

    def get_info(self):
        '''Get detailed dog information.'''
        info = super().get_info()
        info.update({
            'breed': self.breed,
            'owner': self.owner,
            'tricks': self.tricks
        })
        return info
```

- `super().__init__()` calls the parent's constructor to set up name, species, age.
- `make_sound()` overrides the corresponding method on the base class sound with a bark.
- Dog-specific behaviors are added: `fetch`, `learn_trick`, `perform_trick`.

```
# test harness
```

Lecture 11

```
buddy = Dog('Buddy', 'Golden Retriever', 3, 'Alice')

# Use inherited methods
buddy.eat('dog treats')
buddy.sleep()

# Use overridden methods
buddy.make_sound() # Barks

# Use specific methods
buddy.fetch('ball')
buddy.learn_trick('sit')
buddy.learn_trick('roll over')
buddy.perform_trick('sit')

# Access inherited attributes and methods
print(f'Buddy info: {buddy.get_info()}')
```

Another Subclass - Cat

```
class Cat(Animal):
    '''Cat class inheriting from Animal.'''

    def __init__(self, name, breed, age, indoor=True):
        super().__init__(name, 'Felis catus', age)
        self.breed = breed
        self.indoor = indoor
        self.lives_remaining = 9

    def make_sound(self):
        '''Override parent method.'''
        print(f'{self.name} meows: Meow! Meow!')

    def purr(self):
        '''Cat-specific behavior.'''
        print(f'{self.name} is purring contentedly')

    def climb(self, location):
        '''Cat climbing behavior.'''
        print(f'{self.name} climbs up the {location}')

    def get_info(self):
        '''Get detailed cat information.'''
        info = super().get_info()
        info.update({
            'breed': self.breed,
            'indoor': self.indoor,
            'lives_remaining': self.lives_remaining
        })
        return info
```

This time, Cat inherits everything from Animal but:

Lecture 11

- Overrides `make_sound()` with a meow.
- Adds cat-specific methods: `purr()`, `climb()`.
- Introduces an extra attribute: `lives_remaining`.

```
# test harness
whiskers = Cat('Whiskers', 'Persian', 2, indoor=True)

# Use inherited methods
whiskers.eat('fish')
whiskers.sleep()

# Use overridden methods
whiskers.make_sound() # Meows

# Use specific methods
whiskers.purr()
whiskers.climb('cat tree')

# Access inherited attributes and methods
print(f'Whiskers info: {whiskers.get_info()}')
```

Summary

Inheritance is a powerful feature of OOP that:

- Lets you reuse code from a base class.
- Allows extension and customization of behaviors in derived classes.
- Makes programs easier to maintain and reason about.
- Encourages modeling problems in terms of hierarchies.
- Child classes can **reuse**, **override**, or **extend** parent behaviors.
- `super()` is the bridge to call parent methods.

In this notebook, we saw:

1. Basic inheritance with Person → Employee.
2. A shared base class (Animal) with general behaviors.
3. A subclass overriding and extending (Dog).
4. Another specialized subclass (Cat).
5. Three Pillars of Inheritance:
 - Reuse: Leverage existing code from the parent class without rewriting it.
 - Override: Redefine a method from the parent class in the child class to provide specialized behavior (e.g., `make_sound()`).
 - Extend: Add new attributes and methods that are specific to the child class (e.g., `fetch()` for Dog, `purr()` for Cat).
6. Improved Code Structure: By using inheritance, you create a clear, hierarchical, and maintainable codebase where changes in core logic need only be made in one place (the parent class).
7. Maintenance becomes easier and more scalable.

Lecture 11

Together, these examples show how inheritance provides both reuse and flexibility in object-oriented programming.

In the next notebook, we will build upon this by exploring Polymorphism, which works hand-in-hand with inheritance to make our code even more flexible and powerful.

Object Oriented Programming - Encapsulation

```
# The best one so far
# an even more useful class
class Account:
    '''A simple class to represent a bank account with a holder name, account
    number, and balance.
    This class allows you to create an account with a holder's name, an
    automatically generated account number, and an optional balance.

    Attributes:
        __last_number (int): A class variable to keep track of the last assigned
        account number.
        __holder (str): The name of the account holder.
        __number (str): The account number, automatically generated.
        __balance (float): The current balance of the account, defaulting to 0 if
        not specified.

    Methods:
        __init__(name, balance=0): Initializes the account with a holder name, an
        automatically generated account number, and an
        optional balance.
        deposit(amount): Deposits a specified amount into the account.
        withdraw(amount): Withdraws a specified amount from the account.
        __str__(): Returns a string representation of the account, including the
        account number, holder
            name, and balance.
    '''

    __last_number = 1234 # class variable to keep track of the last assigned
    account number

    def __init__(self, name: str, balance: float = 0) -> None:
        '''Initialize the account with a holder name, an automatically
        generated account number, and an optional balance.
```

Lecture 11

```
    Args:
        name (str): The name of the account holder.
        balance (float, optional): The initial balance of the account.
Defaults to 0.

    The double underscore triggers name mangling: Python internally
renames the variable to __Account__name.

    This makes it harder (but not impossible) to access from outside
the class.
'''
self.__holder = name
self.__balance = balance
self.__number = f'AC-{Account.__last_number}'
Account.__last_number += 1

def deposit(self, amount: float) -> None:
    '''Deposit a specified amount into the account.
    Args:
        amount (float): The amount to deposit into the account.

    Raises:
        ValueError: If the amount is negative.
    '''
    if amount < 0:
        raise ValueError('Cannot withdraw a negative amount')
    self.__balance += amount

def withdraw(self, amount: float) -> None:
    '''Deposit a specified amount into the account.
    Args:
        amount (float): The amount to deposit into the account.

    Raises:
        ValueError: If the amount is greater then the balance.
    '''
    if amount > self.balance:
        raise ValueError('You cannot withdraw more than the balance')
    self.__balance -= amount

def __str__(self) -> str:
    '''Return a string representation of this object.
    Returns:
        str: A formatted string containing the account number, holder
name, and balance.
    '''
    return f'[{self.__number}] {self.__holder} ${self.__balance:,.2f}'

@property
def balance(self):
    '''Get the current balance of the account.
    Returns:
        float: The current balance of the account.
    '''
    return self.__balance
```

Lecture 11

```
# test harness
narendra = Account('narendra', 200)
print(acct)

amt = 500
print(f'Deposit ${amt:,.2f}')
narendra.deposit(amt)
amt = -50
try:
    narendra.deposit(amt)
except ValueError as e:
    print(f'Error: {e}')

amt = 150
print(f'Withdraw ${amt:,.2f}')
narendra.withdraw(amt)
print(narendra)

amt = 1000
try:
    narendra.withdraw(amt)
except ValueError as e:
    print(f'Error: {e}')

# acct.balance = 1_000_000
narendra.__balance = 1_000_000
print(narendra)
print(narendra.__balance)

hao = Account('hao', 5_000)
print(hao)
# print(type(narendra))
# print(dir(narendra))
# print(help(narendra))
```